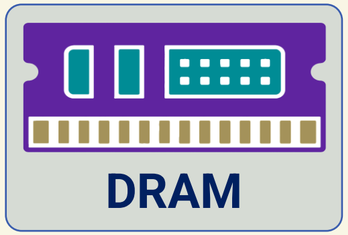
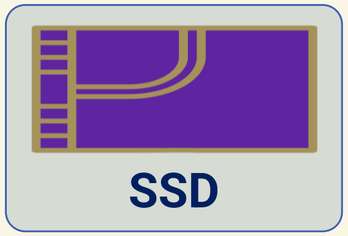
Buffer Manager

Buffer Manager

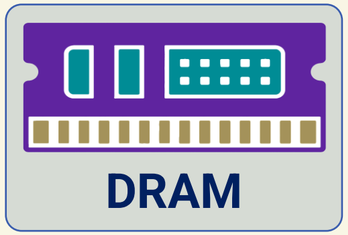
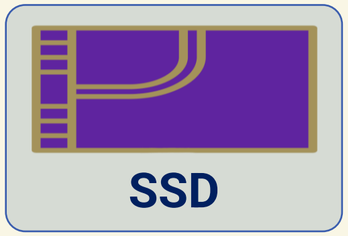
Buffer Manager

Cache Replacement Policy

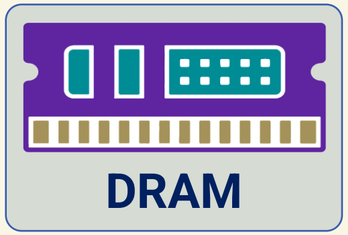
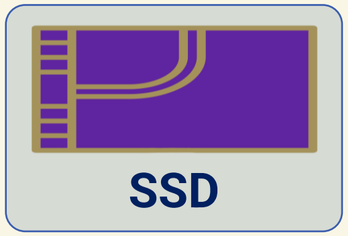
DRAM vs SSD

Device	Latency (ns)	Price per GB ($)
 Dynamic RAM (DRAM)	50	10
 Solid State Disk (SSD)	50 * 1000	0.1

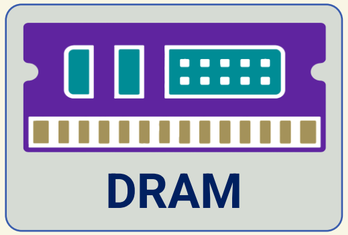
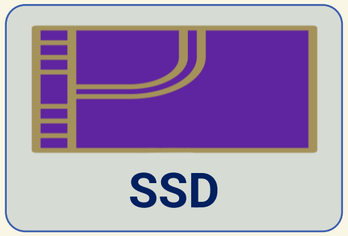
DRAM vs SSD

Device	Latency (ns)	Price per GB ($)
 Dynamic RAM (DRAM)	50	10
 Solid State Disk (SSD)	50 * 1000	0.1

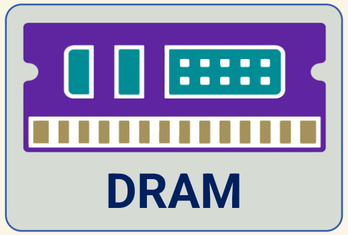
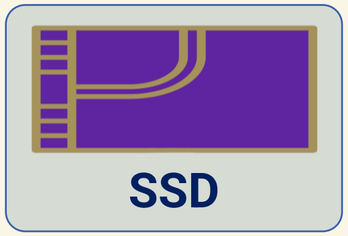
DRAM vs SSD

Device	Latency (ns)	Price per GB ($)
 Dynamic RAM (DRAM)	50	10
 Solid State Disk (SSD)	50 * 1000	0.1

DRAM vs SSD

Device	Latency (ns)	Price per GB ($)
 Dynamic RAM (DRAM)	50	10
 Solid State Disk (SSD)	50 * 1000	0.1

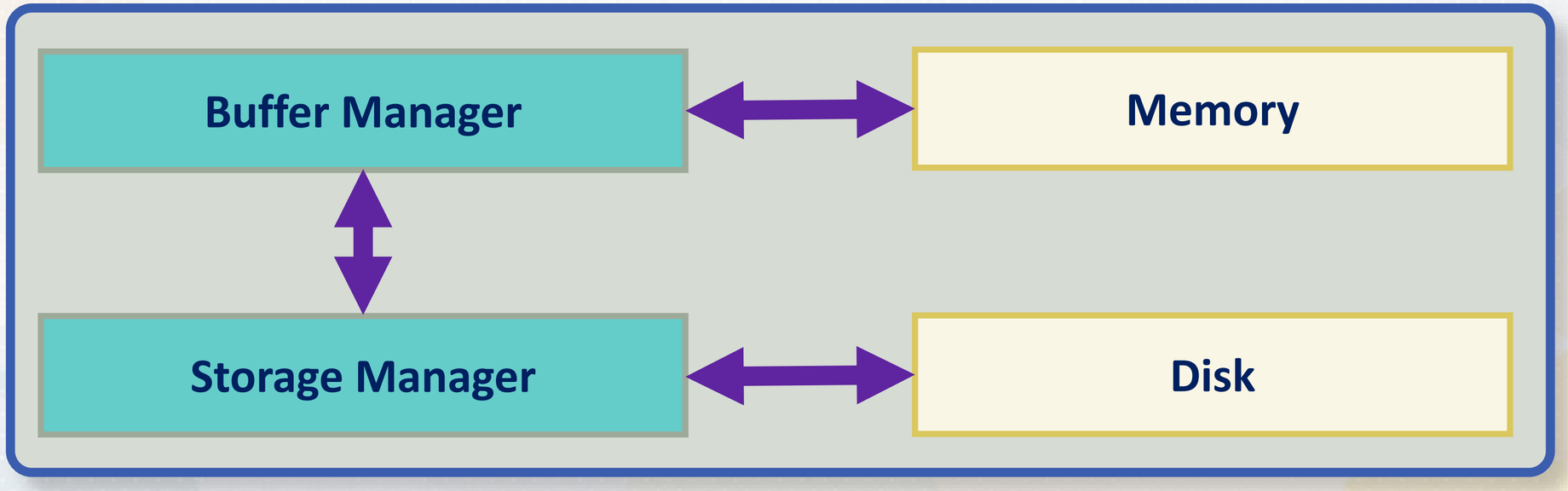
DRAM vs SSD

Device	Latency (ns)	Price per GB ($)
 Dynamic RAM (DRAM)	50	10
 Solid State Disk (SSD)	50 * 1000	0.1

Buffer Manager

Buffer Manager Caches frequently-accessed pages in DRAM

Storage Manager: low-level file I/O operations

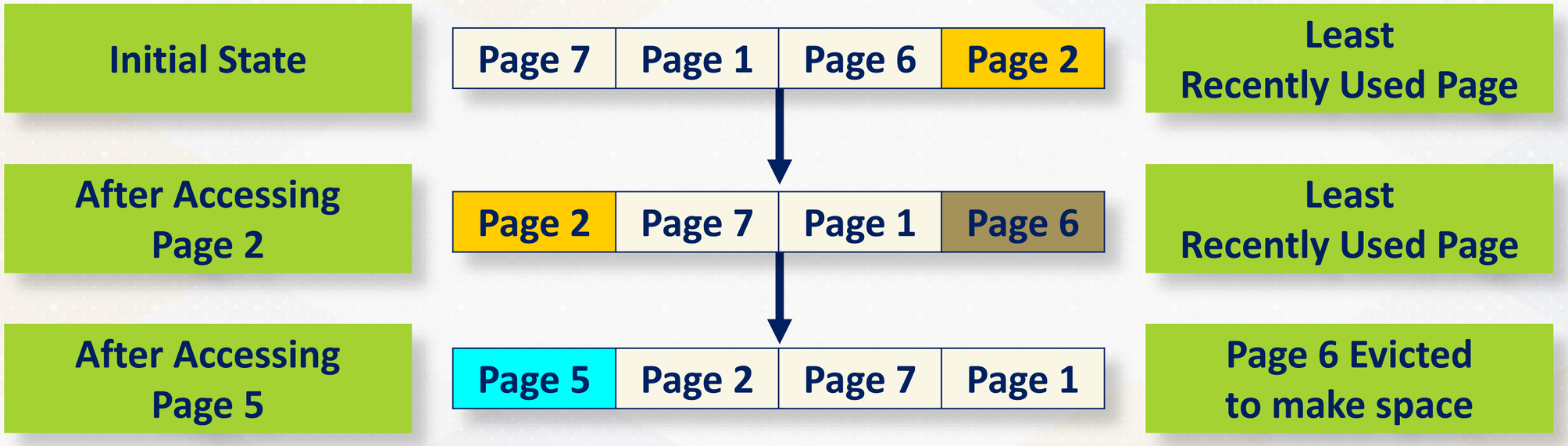


Buffer Manager

Buffer Manager: Maintains a **list of pages** in memory and a mapping from page IDs

```
class BufferManager {
private:
  StorageManager storage_manager;
  // LRU list to maintain the order of page usage
  std::list<std::pair<PageID, std::unique_ptr<SlottedPage>>> lruList;
  // Map to quickly find pages in the list
  std::unordered_map<PageID, typename PageList::iterator> pageMap;
public:
  // Fetches a page with given ID, applying LRU policy for caching
  std::unique_ptr<SlottedPage> &getPage(int page_id);
};
```

Least Recently Used (LRU) Policy



Buffer Manager

```
class BufferManager {
private:
  StorageManager storage_manager;
  // LRU list to maintain the order of page usage
  std::list<std::pair<PageID, std::unique_ptr<SlottedPage>>> lruList;
  // Map to quickly find pages in the list
  std::unordered_map<PageID, typename PageList::iterator> pageMap;
};
```

Buffer Manager

PageID

Utilized for page identification

```
class BufferManager {
private:
  StorageManager storage_manager;
  // LRU list to maintain the order of page usage
  std::list<std::pair<PageID, std::unique_ptr<SlottedPage>>> lruList;
  // Map to quickly find pages in the list
  std::unordered_map<PageID, typename PageList::iterator> pageMap;
};
```

Buffer Manager

PageID

Utilized for page identification

lruList

Stores currently loaded pages in LRU order

```
class BufferManager {
private:
  StorageManager storage_manager;
  // LRU list to maintain the order of page usage
  std::list<std::pair<PageID, std::unique_ptr<SlottedPage>>> lruList;
  // Map to quickly find pages in the list
  std::unordered_map<PageID, typename PageList::iterator> pageMap;
};
```

Buffer Manager

PageID

Utilized for page identification

lruList

Stores currently loaded pages in LRU order

pageMap

mapping from PageID to their locations in lruList

```
class BufferManager {
private:
  StorageManager storage_manager;
  // LRU list to maintain the order of page usage
  std::list<std::pair<PageID, std::unique_ptr<SlottedPage>>> lruList;
  // Map to quickly find pages in the list
  std::unordered_map<PageID, typename PageList::iterator> pageMap;
};
```

Buffer Manager & Storage Manager

StorageManager: Loads Page from Disk when Page Not Found in Buffer Cache

```
std::unique_ptr<SlottedPage> &BufferManager::getPage(int page_id) {
  auto it = pageMap.find(page_id);
  if (it == pageMap.end()) {
    // Page not in cache, load from disk
    auto page = storage_manager.load(page_id);
    ...
  }
  touch(pageMap[page_id]);
  return lruList.begin()->second;
}
```

touch

touch

- Updates a page's position in the **LRU list** to reflect its recent access
- Splice method in **std::list** is used to move the page to the front of the list

```
void touch(PageList::iterator it) {
  // Move page to the front of the list, indicating recent use
  lruList.splice(lruList.begin(), lruList, it);
}
```

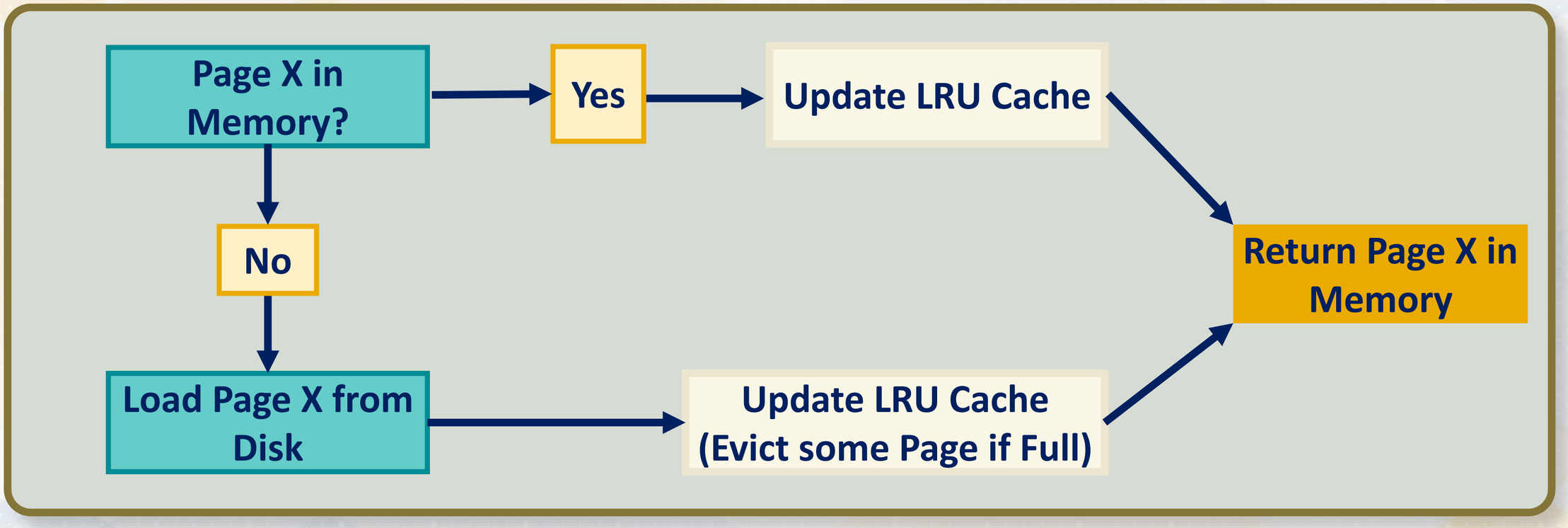
evict

evict

- Removes least recently used page from buffer
- Changes flushed to ensure no data loss

```
void evict() {
  // Get iterator to the last element
  auto last = --lruList.end();
  int evictedPageId = last->first;
  // Flush page changes to disk before eviction if necessary
  storage_manager.flush(evictedPageId, last->second);
  // Remove page from page map and LRU list
  pageMap.erase(evictedPageId);
  lruList.pop_back();
}
```

Buffer Manager



Modularity

buzzDB

- Interacts only with **BufferManager** for page-related operations
- BuzzDB no longer accesses **StorageManager**

```
// BuzzDB now uses BufferManager for page operations
void BuzzDB::insert(int key, int value) {
  ...
  // Interacts through BufferManager
  auto &page = buffer_manager.getPage(page_id);
  page->addTuple(std::move(newTuple));
  ...
}
```

Buffer Manager

Buffer Manager

Cache Replacement Policy

CMYK-Based Tertiaries

IMPACT PURPLE
(dark purple)
PMS: 267 C
RGB: 95, 36, 159
HEX: #5F249F
CMYK: 81, 99, 0, 0

ELECTRIC BLUE
(light green-blue)
PMS: 325 C
RGB: 100 204 201
HEX: #64CCC9
CMYK: 54, 0, 20, 0

RAT CAP
(warm yellow)
PMS: 116 C
RGB: 255, 205, 0
HEX: #FFCD00
CMYK: 0, 10, 98, 0

BOLD BLUE
(medium blue)
PMS: 7455 C
RGB: 58, 93, 174
HEX: #3A5DAE
CMYK: 86, 66, 0, 0

CANOPY LIME
(light warm green)
PMS: 2299 C
RGB: 164, 210, 51
HEX: #A4D233
CMYK: 38, 0, 94, 0

NEW HORIZON
(orange-red)
PMS: 7417 C
RGB: 224, 79, 57
HEX: #E04F39
CMYK: 0, 82, 82, 0

OLYMPIC TEAL
(medium green-blue)
PMS: 321 C
RGB: 0, 140, 149
HEX: #008C95
CMYK: 100, 0, 37, 10

Accessible Tech Gold

Use accessible color combinations to enhance legibility and ensure that Georgia Tech's content is available to a diverse audience. **Text in Tech Gold on a white background is not accessible.** When using Tech Gold for text on a website or other digital media, substitute Tech Medium Gold for large text and Tech Dark Gold for small bold text.

TECH MEDIUM GOLD
HEX: #A4925A
RGB: 164, 146, 90
For use in large headline text in any digital media.

TECH DARK GOLD
HEX: #857437
RGB: 133, 116, 55
For use in small bold text in any digital media.

RGB-Based Tertiaries ("Brights")

The tertiary palette also includes a separate suite of brighter, web-only colors. These color standards maximize visibility and contrast on screens and should not be applied to print projects.

BRIGHT PURPLE
(violet)
RGB: 120, 0, 255
HEX: #7800FF

BRIGHT ELECTRIC
(light teal blue)
RGB: 0, 255, 255
HEX: #00FFFF

BRIGHT BUZZ
(warm yellow)
RGB: 255, 204, 0
HEX: #FFCC00

BRIGHT BLUE
(medium blue)
RGB: 41, 97, 255
HEX: #2961FF

BRIGHT CANOPY
(cool green)
RGB: 0, 236, 156
HEX: #00EC9C

BRIGHT HORIZON
(red-orange)
RGB: 255, 100, 15
HEX: #FF640F

GRAY MATTER
(neutral dark gray)
PMS: 425
HEX: #54585A
RGB: 84, 88, 90
HSL: 200, 3, 34
HSB: 200, 7, 35
CMYK: 48, 29, 26, 76

PI MILE
(light warm gray)
PMS: 427
HEX: #D6DBD4
RGB: 214, 219, 212
HSL: 103, 9, 85
HSB: 103, 3, 86
CMYK: 9, 4, 10, 8

DIPLOMA
(light ivory yellow)
PMS: 7499
HEX: #F9F6E5
RGB: 249, 246, 229
HSL: 51, 63, 94
HSB: 51, 8, 98
CMYK: 1, 1, 10, 1

BUZZ GOLD
PMS: 124
HEX: #EAAA00
RGB: 234, 170, 0
HSL: 44, 100, 46
HSB: 44, 100, 92
CMYK: 0, 30, 100, 0

Cache Eviction Policies

Policy Interface

Policy Interface

Policy vs Mechanism

Policy Interface

Touch and **Evict** functions encapsulate the essence of any eviction policy

```
class Policy {
public:
  virtual void touch(PageID page_id) = 0;
  virtual PageID evict() = 0;
  virtual ~Policy() = default;
};
```

policy interface	Allows for different interchangeable policies

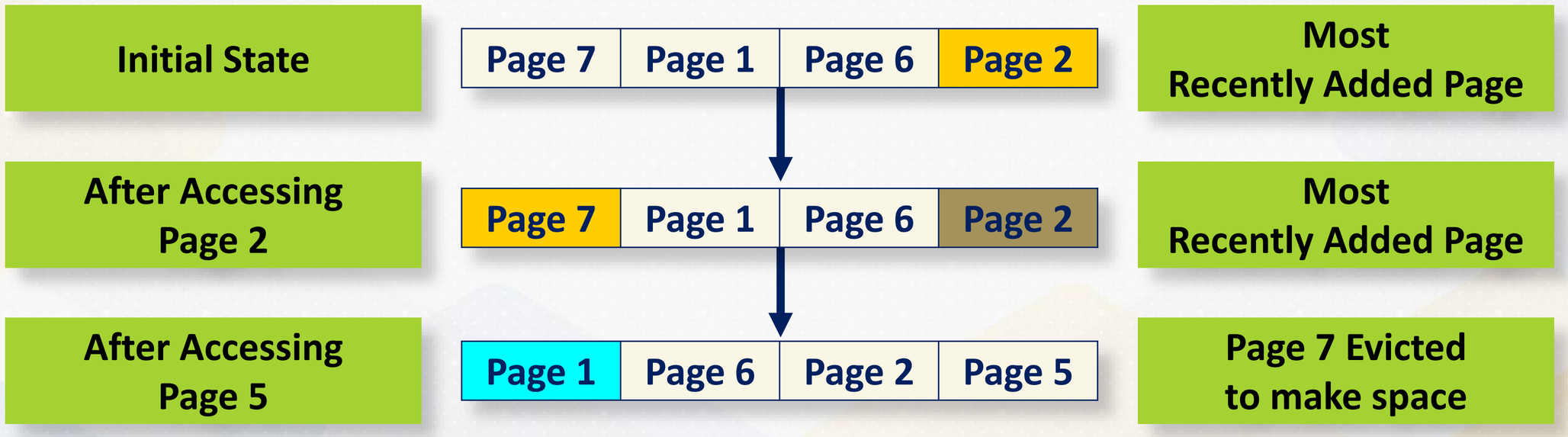
FIFO (First-In-First-Out) Policy

FIFO policy	Evicts the oldest loaded page based on its arrival time

```
class FifoPolicy : public Policy {
  std::queue<PageID> queue;
public:
  void touch(PageID page_id) override {
    auto it = std::find(queue.begin(), queue.end(), page_id);
    if (it == queue.end()) {
      queue.push(page_id);
    }
  }
  PageID evict() override {
    PageID evictedPageId = queue.front();
    queue.pop();
    return evictedPageId;
  }
};
```

FIFO (First-In-First-Out) Policy

FIFO policy	May evict recently accessed pages



LRU (Least Recently Used) Policy

LRU policy	Evicts pages that have not been accessed recently

```
class LruPolicy : public Policy {
  std::list<PageID> lruList;
  std::unordered_map<PageID, std::list<PageID>::iterator> map;

public:
  void
  touch(PageID page_id) override; // Detailed implementation omitted for brevity
  PageID evict() override;        // Evicts the least recently used page
};
```

Integration into Buffer Manager

Buffer Manager	Contains a pointer to the **Policy**

```
class BufferManager {
  std::unique_ptr<Policy> policy;
  // Other members omitted for brevity
public:
  BufferManager(std::unique_ptr<Policy> policy) : policy(std::move(policy)) {}
  // Interface methods omitted for brevity
};
```

Integration into Buffer Manager

touch method | Called to update the **policy state**

```
std::unique_ptr<SlottedPage> &BufferManager::getPage(int page_id) {
  if (it == pageMap.end()) { // Page not in memory
    if (pageMap.size() >= MAX_PAGES_IN_MEMORY) {
      PageID evictedPageId = policy->evict(); // Evict page
      ...
    }
  }
  policy->touch(page_id); // Update policy with recent access
  return pageMap[page_id];
}
```

Evict method invoked if **buffer page limit** is reached

Policy vs. Mechanism

policy	“what” actions should be taken

POLICY (What)	MECHANISM (How)
Cache Eviction Policy (LRU or FIFO etc.)	Cache Eviction Mechanism (Flush page to disk)

mechanism	“how” these actions are taken

Policy vs. Mechanism: Index Maintenance

POLICY (What)	MECHANISM (How)
Lazy vs Eager Index Updates	index[key] = value

Policy Interface

Policy Interface

Policy vs Mechanism

CMYK-Based Tertiaries

IMPACT PURPLE
(dark purple)
PMS: 267 C
RGB: 95, 36, 159
HEX: #5F249F
CMYK: 81, 99, 0, 0

ELECTRIC BLUE
(light green-blue)
PMS: 325 C
RGB: 100 204 201
HEX: #64CCC9
CMYK: 54, 0, 20, 0

RAT CAP
(warm yellow)
PMS: 116 C
RGB: 255, 205, 0
HEX: #FFCD00
CMYK: 0, 10, 98, 0

BOLD BLUE
(medium blue)
PMS: 7455 C
RGB: 58, 93, 174
HEX: #3A5DAE
CMYK: 86, 66, 0, 0

CANOPY LIME
(light warm green)
PMS: 2299 C
RGB: 164, 210, 51
HEX: #A4D233
CMYK: 38, 0, 94, 0

NEW HORIZON
(orange-red)
PMS: 7417 C
RGB: 224, 79, 57
HEX: #E04F39
CMYK: 0, 82, 82, 0

OLYMPIC TEAL
(medium green-blue)
PMS: 321 C
RGB: 0, 140, 149
HEX: #008C95
CMYK: 100, 0, 37, 10

Accessible Tech Gold

Use accessible color combinations to enhance legibility and ensure that Georgia Tech's content is available to a diverse audience. **Text in Tech Gold on a white background is not accessible.** When using Tech Gold for text on a website or other digital media, substitute Tech Medium Gold for large text and Tech Dark Gold for small bold text.

TECH MEDIUM GOLD
HEX: #A4925A
RGB: 164, 146, 90
For use in large headline text in any digital media.

TECH DARK GOLD
HEX: #857437
RGB: 133, 116, 55
For use in small bold text in any digital media.

RGB-Based Tertiaries ("Brights")

The tertiary palette also includes a separate suite of brighter, web-only colors. These color standards maximize visibility and contrast on screens and should not be applied to print projects.

BRIGHT PURPLE
(violet)
RGB: 120, 0, 255
HEX: #7800FF

BRIGHT ELECTRIC
(light teal blue)
RGB: 0, 255, 255
HEX: #00FFFF

BRIGHT BUZZ
(warm yellow)
RGB: 255, 204, 0
HEX: #FFCC00

BRIGHT BLUE
(medium blue)
RGB: 41, 97, 255
HEX: #2961FF

BRIGHT CANOPY
(cool green)
RGB: 0, 236, 156
HEX: #00EC9C

BRIGHT HORIZON
(red-orange)
RGB: 255, 100, 15
HEX: #FF640F

GRAY MATTER
(neutral dark gray)
PMS: 425
HEX: #54585A
RGB: 84, 88, 90
HSL: 200, 3, 34
HSB: 200, 7, 35
CMYK: 48, 29, 26, 76

PI MILE
(light warm gray)
PMS: 427
HEX: #D6DBD4
RGB: 214, 219, 212
HSL: 103, 9, 85
HSB: 103, 3, 86
CMYK: 9, 4, 10, 8

DIPLOMA
(light ivory yellow)
PMS: 7499
HEX: #F9F6E5
RGB: 249, 246, 229
HSL: 51, 63, 94
HSB: 51, 8, 98
CMYK: 1, 1, 10, 1

BUZZ GOLD
PMS: 124
HEX: #EAAA00
RGB: 234, 170, 0
HSL: 44, 100, 46
HSB: 44, 100, 92
CMYK: 0, 30, 100, 0

TwoQ Policy

Buffer Pool Flooding

Buffer Pool Flooding

TwoQ Policy

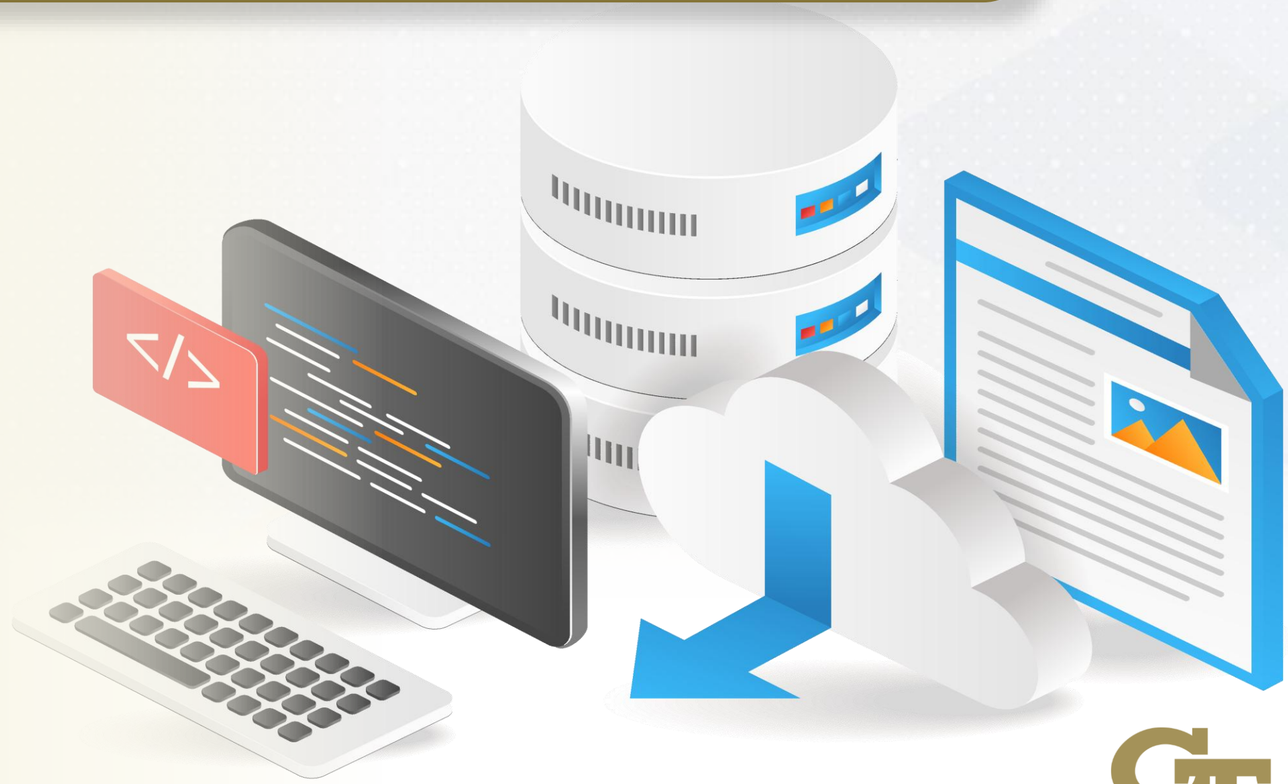
Sequential Scan

sequential scan	when a database reads every table page while processing a query

```
// Sequential scan across the sales_data table
// Assumption: No index on sale_date column
SELECT *
FROM sales_data
WHERE sale_date BETWEEN ‘2040-01-01’ AND ‘2040-01-31’;
```

Buffer Pool Flooding

Sequential scan floods the buffer pool with less important pages

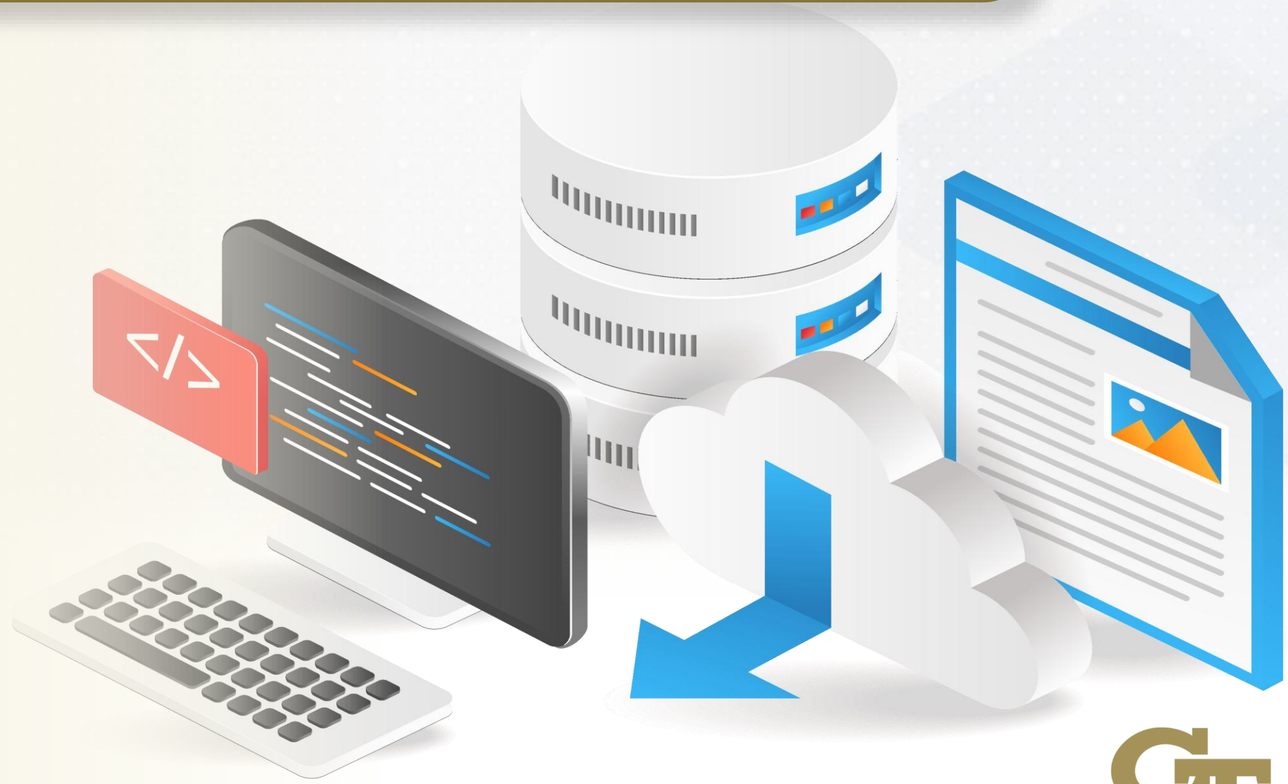


Buffer Pool Flooding

Sequential scan floods the buffer pool with less important pages

FIFO Policy

Evicts the oldest page from cache



Buffer Pool Flooding

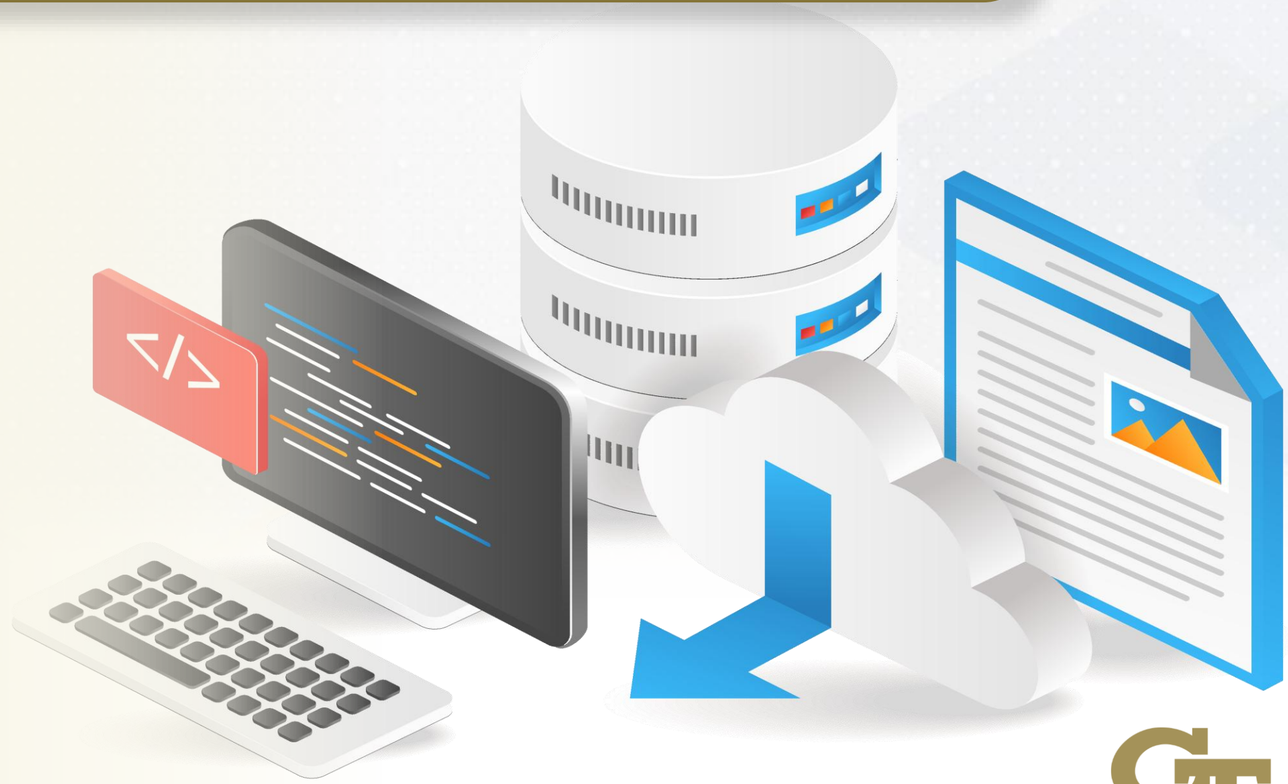
Sequential scan floods the buffer pool with less important pages

FIFO Policy

Evicts the oldest page from cache

LRU Policy

Evicts hot pages to store cold pages



FIFO: Buffer Pool Flooding

Page Access Trace

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
FIFO Queue: 1	1	2	2	2	3	4	1	2	5	6	1	2
FIFO Queue: 2	-	1	1	1	2	3	4	1	2	5	6	1
FIFO Queue: 3	-	-	-	-	1	2	3	4	1	2	5	6
Evict Page						1	2	3	4	1	2	5
Cache Misses	1	2	2	2	3	4	5	6	7	8	9	10

FIFO: Buffer Pool Flooding

Page Access Trace

1 2 1 2 3 4 1 2 5 6 1 2

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
FIFO Queue: 1	1	2	2	2	3	4	1	2	5	6	1	2
FIFO Queue: 2	-	1	1	1	2	3	4	1	2	5	6	1
FIFO Queue: 3	-	-	-	-	1	2	3	4	1	2	5	6
Evict Page						1	2	3	4	1	2	5
Cache Misses	1	2	2	2	3	4	5	6	7	8	9	10

FIFO: Buffer Pool Flooding

Page Access Trace

1 2 1 2 3 4 1 2 5 6 1 2

Buffer Pool Capacity: 3 Pages

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
FIFO Queue: 1	1	2	2	2	3	4	1	2	5	6	1	2
FIFO Queue: 2	-	1	1	1	2	3	4	1	2	5	6	1
FIFO Queue: 3	-	-	-	-	1	2	3	4	1	2	5	6
Evict Page						1	2	3	4	1	2	5
Cache Misses	1	2	2	2	3	4	5	6	7	8	9	10

FIFO: Buffer Pool Flooding

Page Access Trace

1	2	1	2	3	4	1	2	5	6	1	2

Buffer Pool Capacity: 3 Pages

FIFO evicts **hot** pages 1 and 2

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
FIFO Queue: 1	1	2	2	2	3	4	1	2	5	6	1	2
FIFO Queue: 2	-	1	1	1	2	3	4	1	2	5	6	1
FIFO Queue: 3	-	-	-	-	1	2	3	4	1	2	5	6
Evict Page						1	2	3	4	1	2	5
Cache Misses	1	2	2	2	3	4	5	6	7	8	9	10

LRU: Buffer Pool Flooding

Page Access Trace

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
LRU Queue: 1	1	2	1	2	3	4	1	2	5	6	1	2
LRU Queue: 2	-	1	2	1	2	3	4	1	2	5	6	1
LRU Queue: 3	-	-	-	-	1	2	3	4	1	2	5	6
Evict Page						1	2	3	4	1	2	5
Cache Misses	1	2	2	2	3	4	5	6	7	8	9	10

LRU: Buffer Pool Flooding

Page Access Trace

1	2	1	2	3	4	1	2	5	6	1	2

Buffer Pool Capacity: 3 Pages

LRU evicts **hot** pages 1 and 2

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
LRU Queue: 1	1	2	1	2	3	4	1	2	5	6	1	2
LRU Queue: 2	-	1	2	1	2	3	4	1	2	5	6	1
LRU Queue: 3	-	-	-	-	1	2	3	4	1	2	5	6
Evict Page						1	2	3	4	1	2	5
Cache Misses	1	2	2	2	3	4	5	6	7	8	9	10

TwoQ Policy

page transition logic	• Page initially enters **FIFO** queue • Page moves to **LRU** queue on second access

TwoQ Policy

page transition logic	• Page initially enters **FIFO** queue • Page moves to **LRU** queue on second access

FIFO Queue

Read-Once Pages Accessed During Sequential Scan

TwoQ Policy

page transition logic	• Page initially enters **FIFO** queue • Page moves to **LRU** queue on second access

FIFO Queue

Read-Once Pages Accessed During Sequential Scan

LRU Queue

Frequently-Accessed **Hot** Pages Promoted to LRU Queue

TwoQ Policy: touch

```
void touch(PageID page_id) {
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If in FIFO and accessed again, move to LRU
    // Otherwise, adjust position within LRU
    // If cache is full, then evict
    // New pages added to FIFO
  }
}
```

TwoQ Policy: touch

Pages Enter **FIFO** Queue

```
void touch(PageID page_id) {
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If in FIFO and accessed again, move to LRU
    // Otherwise, adjust position within LRU
    // If cache is full, then evict
    // New pages added to FIFO
  }
}
```

TwoQ Policy: touch

Pages Enter **FIFO** Queue

Pages Promoted To **LRU**

```
void touch(PageID page_id) {
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If in FIFO and accessed again, move to LRU
    // Otherwise, adjust position within LRU
    // If cache is full, then evict
    // New pages added to FIFO
  }
}
```

TwoQ Policy: touch

Pages Enter **FIFO** Queue

Pages Promoted To **LRU**

Page Position Updated When Accessed

```
void touch(PageID page_id) {
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If in FIFO and accessed again, move to LRU
    // Otherwise, adjust position within LRU
    // If cache is full, then evict
    // New pages added to FIFO
  }
}
```

TwoQ Policy: touch

```
void touch(PageID page_id) {
  // If the page is already in the cache
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If it's in FIFO, move to LRU
    if (std::find(FIFO.begin(), FIFO.end(), page_id)
          != FIFO.end()) {
      FIFO.erase(it);
      LRU.push_front(page_id);
      pageMap[page_id] = LRU.begin();
    }
    ...
}
```

TwoQ Policy: touch

Check **FIFO** Queue When Page Accessed

```
void touch(PageID page_id) {
  // If the page is already in the cache
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If it's in FIFO, move to LRU
    if (std::find(FIFO.begin(), FIFO.end(), page_id)
          != FIFO.end()) {
      FIFO.erase(it);
      LRU.push_front(page_id);
      pageMap[page_id] = LRU.begin();
    }
    ...
}
```

TwoQ Policy: touch

Check **FIFO** Queue When Page Accessed

Page in FIFO Queue Upgraded to LRU Queue

```
void touch(PageID page_id) {
  // If the page is already in the cache
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If it's in FIFO, move to LRU
    if (std::find(FIFO.begin(), FIFO.end(), page_id)
            != FIFO.end()) {
      FIFO.erase(it);
      LRU.push_front(page_id);
      pageMap[page_id] = LRU.begin();
    }
    ...
}
```

TwoQ Policy: touch

Check **FIFO** Queue When Page Accessed

Page in FIFO Queue Upgraded to LRU Queue

Page in LRU Queue Has Higher Reuse Probability

```
void touch(PageID page_id) {
  // If the page is already in the cache
  if (pageMap.find(page_id) != pageMap.end()) {
    auto it = pageMap[page_id];
    // If it's in FIFO, move to LRU
    if (std::find(FIFO.begin(), FIFO.end(), page_id)
            != FIFO.end()) {
      FIFO.erase(it);
      LRU.push_front(page_id);
      pageMap[page_id] = LRU.begin();
    }
    ...
}
```

TwoQ Policy: touch


```
void touch(PageID page_id){
  else {
     // If it's in LRU, move to the front
     LRU.erase(it);
     LRU.push_front(page_id);
     pageMap[page_id] = LRU.begin();
  }
  ...
}
```

TwoQ Policy: touch

```
void touch(PageID page_id) {
  ...
  // If the page is not in the cache
  else {
    // If cache is full, evict
    if (FIFO.size() + LRU.size() >= cacheSize) {
      evict();
    }
    // Add page to FIFO
    FIFO.push_back(page_id);
    pageMap[page_id] = std::prev(FIFO.end());
  }
}
```

TwoQ Policy: touch

Check if Cache is Full

```
void touch(PageID page_id) {
  ...
  // If the page is not in the cache
  else {
    // If cache is full, evict
    if (FIFO.size() + LRU.size() >= cacheSize) {
      evict();
    }
    // Add page to FIFO
    FIFO.push_back(page_id);
    pageMap[page_id] = std::prev(FIFO.end());
  }
}
```

TwoQ Policy: touch

Check if Cache is Full

Sum of size of
FIFO and LRU Queues

```
void touch(PageID page_id) {
  ...
  // If the page is not in the cache
  else {
    // If cache is full, evict
    if (FIFO.size() + LRU.size() >= cacheSize) {
      evict();
    }
    // Add page to FIFO
    FIFO.push_back(page_id);
    pageMap[page_id] = std::prev(FIFO.end());
  }
}
```

TwoQ Policy: touch

Check if Cache is Full

Sum of size of
FIFO and LRU Queues

Evict Method Triggered
if Cache is Full

```
void touch(PageID page_id) {
  ...
  // If the page is not in the cache
  else {
    // If cache is full, evict
    if (FIFO.size() + LRU.size() >= cacheSize) {
      evict();
    }
    // Add page to FIFO
    FIFO.push_back(page_id);
    pageMap[page_id] = std::prev(FIFO.end());
  }
}
```

TwoQ Policy: evict

TwoQ evict	• Prioritizes Eviction from **FIFO** Queue • Minimizes Cache Pollution from Sequential Scans

```
PageID evict() {
  // FIFO pages are evicted first to minimize cache pollution,
  // followed by the least recently used pages in LRU
}
```

TwoQ Policy: evict

```
PageID evict() {
  PageID evictedPageId = INVALID_VALUE;
  if (!FIFO.empty()) {
    evictedPageId = FIFO.front();
    FIFO.pop_front();
    pageMap.erase(evictedPageId);
  } else if (!LRU.empty()) {
    evictedPageId = LRU.back();
    LRU.pop_back();
    pageMap.erase(evictedPageId);
  }
  return evictedPageId;
}
```

TwoQ Policy: evict

FIFO Eviction Targets Oldest Page in Cache

```
PageID evict() {
  PageID evictedPageId = INVALID_VALUE;
  if (!FIFO.empty()) {
    evictedPageId = FIFO.front();
    FIFO.pop_front();
    pageMap.erase(evictedPageId);
  } else if (!LRU.empty()) {
    evictedPageId = LRU.back();
    LRU.pop_back();
    pageMap.erase(evictedPageId);
  }
  return evictedPageId;
}
```

TwoQ Policy: evict

FIFO Eviction Targets Oldest Page in Cache

Last-Used Page in **LRU Cache** Evicted if **FIFO Cache** Empty

```
PageID evict() {
  PageID evictedPageId = INVALID_VALUE;
  if (!FIFO.empty()) {
    evictedPageId = FIFO.front();
    FIFO.pop_front();
    pageMap.erase(evictedPageId);
  } else if (!LRU.empty()) {
    evictedPageId = LRU.back();
    LRU.pop_back();
    pageMap.erase(evictedPageId);
  }
  return evictedPageId;
}
```

TwoQ: No Buffer Pool Flooding

TwoQ retains **hot** pages like 1 and 2

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
FIFO Queue:1	1	2	2	_	3	4	4	4	5	6	6	6
FIFO Queue:2	-	1	-	-	-	-	-	-	-	-	-	-
FIFO Queue:3	-	-	-	-	-	-	-	-	-	-	-	-
LRU Queue: 1	-	-	1	2	2	2	1	2	2	2	1	2
LRU Queue: 2	-	-	-	1	1	1	2	1	1	1	2	1
LRU Queue: 3	-	-	-	-	-	-	-	-	-	-	-	-
Evict Page						3			4	5		
Cache Misses	1	2	2	2	3	4	4	4	5	6	6	6

TwoQ: No Buffer Pool Flooding

TwoQ retains **hot** pages like 1 and 2

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
FIFO Queue:1	1	2	2	_	3	4	4	4	5	6	6	6
FIFO Queue:2	-	1	-	-	-	-	-	-	-	-	-	-
FIFO Queue:3	-	-	-	-	-	-	-	-	-	-	-	-
LRU Queue: 1	-	-	1	2	2	2	1	2	2	2	1	2
LRU Queue: 2	-	-	-	1	1	1	2	1	1	1	2	1
LRU Queue: 3	-	-	-	-	-	-	-	-	-	-	-	-
Evict Page						3			4	5		
Cache Misses	1	2	2	2	3	4	4	4	5	6	6	6

TwoQ: No Buffer Pool Flooding

TwoQ retains **hot** pages like 1 and 2

Access Page	1	2	1	2	3	4	1	2	5	6	1	2
FIFO Queue:1	1	2	2	_	3	4	4	4	5	6	6	6
FIFO Queue:2	-	1	-	-	-	-	-	-	-	-	-	-
FIFO Queue:3	-	-	-	-	-	-	-	-	-	-	-	-
LRU Queue: 1	-	-	1	2	2	2	1	2	2	2	1	2
LRU Queue: 2	-	-	-	1	1	1	2	1	1	1	2	1
LRU Queue: 3	-	-	-	-	-	-	-	-	-	-	-	-
Evict Page						3			4	5		
Cache Misses	1	2	2	2	3	4	4	4	5	6	6	6

More Policies: LFU

More Policies: LFU

LFU Policy Benefits

Keeps Frequently-Used Pages in Memory

More Policies: LFU

LFU Policy Benefits

Keeps Frequently-Used Pages in Memory

LFU Policy Limitations

Does Not Consider Changes to Recent Access Patterns

More Policies: ARC

More Policies: ARC

ARC Policy Benefits

Dynamically Balances between LRU and LFU

More Policies: ARC

ARC Policy Benefits

Dynamically Balances between LRU and LFU

ARC Policy Limitations

Complex Implementation

Music Playlist + Cache Eviction Policies

LRU	Keeps Only New Discoveries
TwoQ	Keeps Favorites and New Discoveries

Buffer Pool Flooding

Buffer Pool Flooding

TwoQ Policy

CMYK-Based Tertiaries

IMPACT PURPLE
(dark purple)
PMS: 267 C
RGB: 95, 36, 159
HEX: #5F249F
CMYK: 81, 99, 0, 0

ELECTRIC BLUE
(light green-blue)
PMS: 325 C
RGB: 100 204 201
HEX: #64CCC9
CMYK: 54, 0, 20, 0

RAT CAP
(warm yellow)
PMS: 116 C
RGB: 255, 205, 0
HEX: #FFCD00
CMYK: 0, 10, 98, 0

BOLD BLUE
(medium blue)
PMS: 7455 C
RGB: 58, 93, 174
HEX: #3A5DAE
CMYK: 86, 66, 0, 0

CANOPY LIME
(light warm green)
PMS: 2299 C
RGB: 164, 210, 51
HEX: #A4D233
CMYK: 38, 0, 94, 0

NEW HORIZON
(orange-red)
PMS: 7417 C
RGB: 224, 79, 57
HEX: #E04F39
CMYK: 0, 82, 82, 0

OLYMPIC TEAL
(medium green-blue)
PMS: 321 C
RGB: 0, 140, 149
HEX: #008C95
CMYK: 100, 0, 37, 10

Accessible Tech Gold

Use accessible color combinations to enhance legibility and ensure that Georgia Tech's content is available to a diverse audience. **Text in Tech Gold on a white background is not accessible.** When using Tech Gold for text on a website or other digital media, substitute Tech Medium Gold for large text and Tech Dark Gold for small bold text.

TECH MEDIUM GOLD
HEX: #A4925A
RGB: 164, 146, 90
For use in large headline text in any digital media.

TECH DARK GOLD
HEX: #857437
RGB: 133, 116, 55
For use in small bold text in any digital media.

RGB-Based Tertiaries ("Brights")

The tertiary palette also includes a separate suite of brighter, web-only colors. These color standards maximize visibility and contrast on screens and should not be applied to print projects.

BRIGHT PURPLE
(violet)
RGB: 120, 0, 255
HEX: #7800FF

BRIGHT ELECTRIC
(light teal blue)
RGB: 0, 255, 255
HEX: #00FFFF

BRIGHT BUZZ
(warm yellow)
RGB: 255, 204, 0
HEX: #FFCC00

BRIGHT BLUE
(medium blue)
RGB: 41, 97, 255
HEX: #2961FF

BRIGHT CANOPY
(cool green)
RGB: 0, 236, 156
HEX: #00EC9C

BRIGHT HORIZON
(red-orange)
RGB: 255, 100, 15
HEX: #FF640F

GRAY MATTER
(neutral dark gray)
PMS: 425
HEX: #54585A
RGB: 84, 88, 90
HSL: 200, 3, 34
HSB: 200, 7, 35
CMYK: 48, 29, 26, 76

PI MILE
(light warm gray)
PMS: 427
HEX: #D6DBD4
RGB: 214, 219, 212
HSL: 103, 9, 85
HSB: 103, 3, 86
CMYK: 9, 4, 10, 8

DIPLOMA
(light ivory yellow)
PMS: 7499
HEX: #F9F6E5
RGB: 249, 246, 229
HSL: 51, 63, 94
HSB: 51, 8, 98
CMYK: 1, 1, 10, 1

BUZZ GOLD
PMS: 124
HEX: #EAAA00
RGB: 234, 170, 0
HSL: 44, 100, 46
HSB: 44, 100, 92
CMYK: 0, 30, 100, 0